국문제목

소형 로컬 LLM에서 Hermes Agent 메모리의 반복 압축이 일으키는 정보 손실 분석과 변경 및 예외를 고려한 결정론적 병합

영문제목

Information Loss from Repeated Memory Compression in Hermes Agent with Small Local LLMs and a Change- and Exception-Aware Deterministic Merge

요 약

Hermes Agent는 일정한 크기의 영구 메모리를 LLM이 스스로 정리하고, 대화가 길어지는 경우 이전 요약에 새 대화를 덧붙여 반복 요약함으로써, 대화 내용을 기억하고, 사용자의 선호를 학습하며, 같은 종류의 작업을 반복하면 점점 빨라지는 오픈소스 AI 에이전트이다. 본 연구는 소형 로컬 모델(Qwen3.5 4B, 9B)로 Hermes Agent의 메모리 도구와 문맥 압축기를 구현하여, 정답이 라벨링된 475개의 질문에서의 정보 손실을 측정하였다. 9B 모델은 유효 사실의 30%만 보존하였으며, 정확도는 1회 압축 후 68.2%에서 10회 압축 후 52.2%까지 떨어졌다. 이를 개선하기 위해 본 연구는 LLM에게는 사실 추출만 맡기고 병합 및 압축은 (엔티티, 속성, 범위) 키 기반의 결정적 규칙으로 처리하는 CEM을 제안한다. CEM은 2,200자에서 정확도 64.0%로 LLM 요약(51.4%)과 Hermes Agent(30.3%)보다 높았고, 틀린 응답은 LLM 요약의 절반 이하였으며, 반복 압축에도 손실이 쌓이지 않았다.

1. 서 론

LLM 에이전트를 오래 운용하면 누적되는 대화와 기억을 제한된 공간에 담기 위해 압축을 반복하게 된다. 오픈소스 에이전트인 Hermes Agent[1]가 그 예이다. Hermes Agent는 영구 메모리 파일인 MEMORY.md의 크기를 2,200자로 제한하며, 이 한도를 넘으면 세션마다 LLM이 기존 항목을 합치거나 삭제한다. 대화가 길어질 때는 이전 요약에 새 대화를 덧붙인 뒤 이를 다시 요약한다. 두 방식 모두 LLM이 자신이 쓴 요약을 다시 고쳐 쓰는 구조이므로 한 번 누락되거나 왜곡된 정보는 이후 단계에서 영구적으로 소실된다.

이 문제는 시간에 따라 값이 바뀌거나 일부 범위에만 예외가 적용되는 설정 정보에서 특히 심각하다. 예를 들어 서버 포트가 8080에서 9090으로 변경되었는데 압축 후에도 옛 값이 남을 수 있다. 또는 staging 환경에만 적용되는 8081이 전체 기본값으로 기록될 수 있다. 이런 경우 메모리는 틀린 정보를 현재의 사실처럼 유지하게 된다. Hermes Agent도 여러 사실을 담은 항목이 통째로 덮어써지는 문제 때문에 replace 동작을 제거하였다. 또한 30B 미만 모델로 메모리 도구를 사용하는 것에 대해 주의를 권고하고 있다[1]. 그러나 소형 로컬 모델 환경에서 반복 압축으로 어떤 정보가 얼마나 손실되는지를 실제 배포 코드로 측정한 연구는 찾아보기 어렵다.

이에 본 연구에서는 Hermes Agent의 실제 코드를 그대로 사용하여 반복 압축에 따른 정보 손실을 측정하였다. 손실은 정보 유형별, 압축 횟수별로 측정하였다. 또한 결정적 병합 기법인 CEM(Change- and Exception-aware Memory)을 제안하였다. CEM은 LLM에게 사실 추출만 맡기고, 병합은 정해진 규칙에 따라 수행한다. 마지막으로 제거 실험(ablation study)을 통해 각 요소가 성능에 기여하는 정도를 확인하였다.

2. 관련 연구

MemGPT[2]는 LLM이 스스로 메모리를 관리하는 방식을 제안하였다. 이후 Mem0[3]는 대화에서 추출한 사실마다 LLM이 추가, 수정, 삭제 중 하나를 선택하도록 하였다. Zep[4]은 각 사실에 유효 기간을 부여한 그래프 구조를 사용하여 더 이상 유효하지 않은 사실을 무효화한다. 평가에서는 LongMemEval[5]이 지식 갱신을 별도의 평가 항목으로 포함하였다.

최근에는 메모리 관리 과정에서 발생하는 손실을 다룬 연구도 늘고 있다. 보고된 현상으로는 갱신된 정보의 소실[6], 반복 통합으로 인한 기억 오염[7], 요약 과정에서의 조건 누락[8], 압축을 거듭할 때 손실이 급격히 커지는 현상[9]이 있다. 대응 방법으로는 근거 추출과 갱신을 분리하고 갱신을 결정적으로 처리하는 방법[10, 11]이 제안되었다. 다만 이들 연구는 대부분 검색 기반 메모리와 대형 모델을 대상으로 한다. 본 연구는 두 가지 점에서 이들과 다르다. 첫째, 글자 수 예산이 고정된 메모리와 반복적인 문맥 압축을 소형 로컬 모델로 운용하는 상황에 초점을 맞춘다. 둘째, 동일한 추출 결과를 입력으로 하여 Mem0 방식의 LLM 병합과 결정적 병합을 직접 비교한다.

3. 제안 방법: CEM

그림 1은 Hermes Agent와 CEM을 비교한 것이다. CEM은 정보 추출, 근거 검증, 결정론적 병합, 예산 배치의 네 단계로 동작한다. 첫째로, LLM이 세션 메모마다 (엔티티, 속성, 범위, 값, 이전값)을 JSON 스키마 제약 출력으로 추출하고, 둘째로, 원문에 나타나지 않는 값은 버린다. 셋째로, 엔티티, 속성, 범위 이름을 정규화하여 (엔티티, 속성, 범위) 키마다 시간순 버전 사슬을 만든다. 마지막으로, 엔티티별 정보를 "Orion: API port 9090

(was 8080; staging: 8081)" 형태의 Hermes Agent 항목 하나로 렌더링하고, 글자 수가 적은 정보 순으로 추가하여 글자 수 대비 정보량을 최대화한다. LLM은 메모리 본문을 다시 쓰지 않고 버전 사슬 파일을 매번 같은 규칙으로 렌더링하므로, 결과가 압축 횟수와 무관하다.

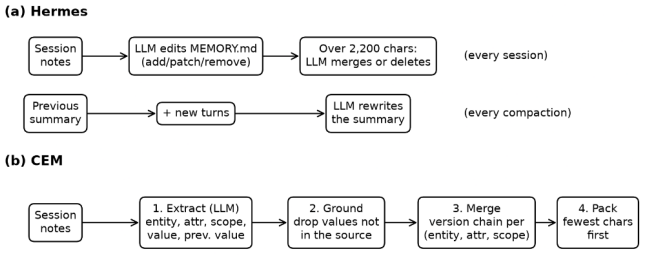


그림 1 Hermes Agent의 기본 반복 압축(a: 메모리 통합과 문맥 압축)과 제안 방법 CEM(b)

4. 실험 설계

4.1 데이터와 평가

영어 템플릿으로 엔티티 24개, 엔티티당 속성 3개로 이루어진 합성 데이터를 만들었다(날짜별 파일 37~49개, 원문 1.0~1.3만 자). 각 속성은 50% 확률로 값이 바뀌고, 35% 확률로 범위 예외가, 12% 확률로 기각된 제안이 생기며, 메모에 속성 이름을 쓸 때마다 30% 확률로 동의어를 쓴다. 값은 에피소드 안에서 유일하게 존재하므로 오답이 어디서 발생한 것인지 자동으로 판정할 수 있다. 질문은 '현재값, 불변값, 직전값, 예외값, 예외 없는 범위, 자유 메모, 모름'의 7가지 유형이며, 독립 에피소드 3개(시드 1~3)에서 모두 475개이다. 같은 모델이 각 메모리만 보고 응답하도록 하였으며, '정확도'는 맞힌 문항의 비율, '매크로 정확도'는 유형별 정확도의 평균, '오답률'은 정답이 있는 질문 중 '모름'이 아닌 틀린 값을 답한 비율이다.

4.2 실험 1: 2,200자 메모리

메모리 예산은 2,200자이며 길이는 Hermes Agent와 같은 방식으로 계산하였다. 비교 방법은 최신 노트 유지, 한도를 초과하는 추가를 거부하는 Hermes Agent 무통합, LLM 요약, CEM과 같은 출력 형식의 구조화 요약, 실제 메모리 도구와 지침을 사용한 Hermes Agent와 그 JSON 강제 변형이 있으며, 무압축 원문과 BM25 검색을 참조의 기준으로 하였다. LLM을 쓰는 방법에는 모두 같은 평가 안내문을 주었다. 제거 실험으로는 이전값 보존 제거, 범위 키 제거, 같은 추출 결과의 LLM 병합(Mem0 방식), 정답 추출(Oracle)을 평가하였다.

4.3 실험 2: 반복 문맥 압축

기록을 시간순으로 k개(k = 1, 3, 5, 10) 구간으로 나누고, Hermes Agent 문맥 압축기의 실제 갱신 프롬프트로 이전 요약과 다음 구간을 함께 요약하는 과정을 반복하였다. 이때, 요약 예산은 Hermes Agent의 기본값인 2,000토큰이다. 비교를 위해, 매번 원문 전체를 새로 요약하는 원본 참조 재압축과, 글자 제한 없이 렌더링한 CEM 원본 파일을 함께 평가하였다.

4.4 실험 환경

RTX 3060, Ollama, qwen3.5:9b와 qwen3.5:4b를 사용했고, Hermes 커밋 67481b6의 코드를 수정 없이 사용하였다. 유의성은 McNemar 검정(Holm 보정)과 군집 부트스트랩 95% 신뢰구간으로 검정하였다.

5. 실험 결과

5.1 실험 1: 2,200 자

표 1에서 9B의 CEM 정확도는 64.0%로 LLM 요약(51.4%), 구조화 요약(56.4%), Hermes 에이전트(30.3%), 최신 노트 유지(25.7%)보다 높았다. 매크로 정확도 차이는 LLM 요약 대비 +10.5%p(95% CI +2.4~+18.4), 구조화 요약 대비 +9.9%p(+1.9~+17.7)였고, McNemar 검정에서도 유의미했다(Holm 보정 p=0.0002, 0.035). CEM의 오답률은 7.3%로 LLM 요약(16.9%)과 무압축 원문(17.8%)의 절반 이하였고, 직전값 정확도는 59.6%로 무압축 원문(46.8%)보다도 높았다. 원문을 보관하는 무압축 원문과 BM25(약 78%)는 상한 역할을 했다. 4B(CEM 64.0%, LLM 요약 58.7%)와 개발에 쓰지 않은 시드 2~3(9B, CEM 62.7%, LLM 요약 47.1%)에서도 CEM이 가장 높았다.

표 1 2,200자 메모리에서 방법별 정확도와 오답률(%, 9B, 시드 1~3, 475문항)

방법	현재	직전	예외	전체	매크로	오답률	질의 토큰
Full	93.5	46.8	87.2	78.7	77.6	17.8	5600
BM25	93.1	39.4	89.7	77.9	78.6	17.3	1052
Recency	21.3	9.2	32.1	25.7	29.8	11.8	1100
NoConsol	23.1	16.5	1.3	23.6	29.4	13.9	1176
Summary	70.4	6.4	52.6	51.4	48.8	16.9	1114
StructSum	77.3	20.2	48.7	56.4	49.4	17.8	1287
Hermes-tool	29.6	13.8	26.9	30.3	35.1	8.9	845
Hermes-JSON	38	13.8	32.1	34.7	38.2	17.1	1016
CEM	70.8	59.6	50	64	59.3	7.3	1170

5.2 손실의 종류

Hermes Agent는 한 세션에서 최대 1,437자를 줄였고, 그 시점에서의 기본값과 예외값 등 유효 사실의 보존율은 마지막에 30%까지 떨어졌다. 반면 CEM은 71%를 유지하였다(그림 2). 도구 호출의 50%가 실패했고, JSON 출력을 강제하여 형식 오류를 없애도 정확도는 34.7%로 소형 모델의 통합 정책 자체가 손실의 원인으로 보인다. 최신 노트 유지와 Hermes 계열의 오류는 대부분 누락이었던 반면, LLM 요약 계열은 버전 혼동(약 10%)과 범위 혼동이 가장 많았고 CEM은 각각 4.2%, 2.1%였다.

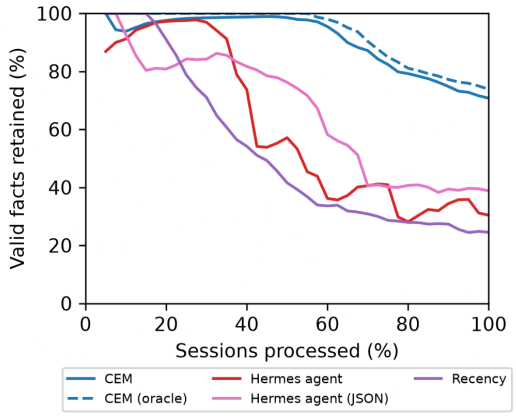


그림 2 세션 진행에 따른 유효 사실 보존율(9B, 시드 1~3 평균). 파선은 정답 추출(Oracle)을 쓴 CEM이다.

5.3 제거 실험

제거 실험 결과는 표 1의 아래 네 행이다. 같은 추출 결과를 LLM이 병합하면 정확도가 27.6%로 떨어졌는데, 글자 수 제한에 맞춰 자르기 전에도 LLM 병합본에는 정보의 68~85%(4B의 경우 34~47%)만 남았고 결론정적 병합은 95~98%를 보존하였다. 범위 키를 빼면 범위 혼동이 2.1%에서 14.9%로 늘었고, 이전값을 빼면 직전값 정확도가 59.6%에서 1.8%로 떨어졌다. 정답 추출과의 정확도 차이는 4.0%p로 나타났다. 동의어 처리에서 동의어가 한 번 이상 쓰인 키의 정확도는 58.9%로 그렇지 않은 키(67.1%)보다 낮았다.

5.4 실험 2: 반복 문맥 압축

Hermes 방식의 요약본 재압축에서 정확도는 k=1, 3, 5, 10일 때 4B가 62.1, 62.5, 54.7, 45.1%, 9B가 68.2, 69.5, 60.2, 52.2%로 k=3 이후로 떨어졌다(그림 3).

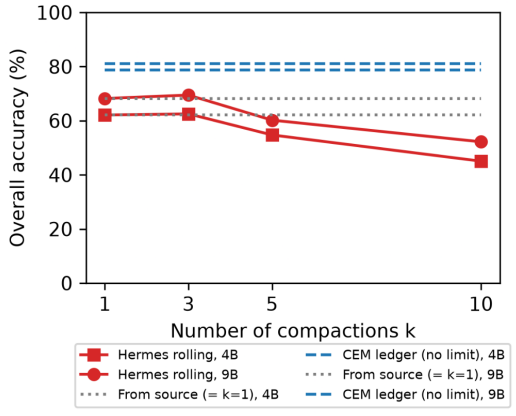


그림 3 압축 횟수 k에 따른 전체 정확도(4B, 9B, 시드 1~3). 실선은 Hermes 방식의 요약본 재압축(네모 4B, 동그라미 9B), 점선은 원본 참조 재압축(k=1 값), 파선은 글자 제한이 없는 CEM 원본(위쪽이 4B)이다.

반복할수록 9B에서는 예외 조건의 정확도가 74.4%에서 24.4%로, 4B에서는 현재 설정값의 정확도가 80.6%에서 41.8%로 떨어졌고, 옛 값이 요약에 남아있어도 모델은 그 값이 직전 값인지 구별하지 못했다. 원본 참조 재압축은 정확도가 k=1과 같지만, 매번 원문 전체를 다시 읽어야 하므로 압축이 필요한 상황에서는 사용하지 못한다. CEM 원장은 k와 관계없이 4B 81.1%, 9B 78.7%였고, 길이는 약 3,500자로 Hermes 요약의 40% 수준이었다.

6. 결 론

Hermes Agent의 반복 압축을 소형 로컬 LLM으로 수행한 결과, 대량 삭제, 직전값과 예외값의 혼동, 반복에 따른 누적 손실이 나타났다. LLM을 정보 추출에만 사용하고 병합과 압축을 결정적으로 수행한 CEM은 동일한 글자 수 제한에서 정확도를 높이고 틀린 값 응답을 절반 이하로 줄였으며, 반복 압축에서도 성능을 유지하였다. 다만 본 연구는 영어로 생성한 합성 데이터와 두 모델을 사용하였으며, Hermes Agent의 메모리 관리 및 문맥 압축 기능만 평가하였다. 정보 추출 설정은 시드 1의 결과를 바탕으로 수정한 후, 모든 시드에 대해 실험을 다시 수행하였다. 또한 같은 속성을 뜻하는 서로 다른 표현을 별개의 항목으로 저장하는 문제가 남아 있다. 향후 실제 사용 기록과 한국어 메모를 대상으로 평가를 확대할 계획이다.

참고문헌

[1] Nous Research, "Hermes Agent: Persistent Memory and Context Compression," GitHub repository, commit 67481b6, 2026. https://github.com/NousResearch/hermes-agent
[2] C. Packer et al., "MemGPT: Towards LLMs as Operating Systems," arXiv preprint arXiv:2310.08560, 2023.
[3] P. Chhikara et al., "Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory," Proc. of ECAI 2025, Frontiers in Artificial Intelligence and Applications, Vol. 413, pp. 2993-3000, 2025.
[4] P. Rasmussen et al., "Zep: A Temporal Knowledge Graph Architecture for Agent Memory," arXiv preprint arXiv:2501.13956, 2025.
[5] D. Wu et al., "LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory," Proc. of ICLR 2025, 2025.
[6] V. Patel, "Supersede: Diagnosing and Training the Memory-Update Gap in LLM Agents," arXiv preprint arXiv:2606.27472, 2026.
[7] D. Zhang et al., "Useful Memories Become Faulty When Continuously Updated by LLMs," arXiv preprint arXiv:2605.12978, 2026.
[8] I. Garg et al., "MemFail: Stress-Testing Failure Modes of LLM Memory Systems," arXiv preprint arXiv:2605.26667, 2026.
[9] S. Zerhoudi et al., "The Compaction Cliff in Long-Running AI Agent Memory," Proc. of the 35th ACM CIKM, 2026 (arXiv:2608.22752).
[10] V. Reddy, S. R. Challaram, "Reliable Post-Retrieval Assembly for Agent Memory: Separating Evidence Extraction from Policy Execution," Lifelong Agent Workshop at COLM 2026, arXiv:2606.01435, 2026.
[11] N. Yadav, "Temporal Validity in Retrieval Memory: Eliminating Stale-Fact Errors for AI Agents over Evolving Knowledge," arXiv preprint arXiv:2606.26511, 2026.